PROJECT REPORT

FACULTY LEAVE MANAGEMENT SYSTEM

A C++ Object-Oriented Programming Project

Student Name	Hiya Solanki
Enrollment Number	IU2541230522
Project Code	CPP56
Programming Language	C++
Development Environment	Visual Studio Code with GNU G++ (MSYS2 UCRT64)

Submitted as part of the C++ / Object-Oriented Programming project work.

Academic Project Report • October 2026

Table of Contents

Section	Topic
1	Project Overview
2	Problem Statement
3	Objectives and Scope
4	Features of the System
5	System Requirements and Tools
6	System Design and Program Workflow
7	Classes and Responsibilities
8	Object-Oriented Programming Concepts
9	File Handling and Data Persistence
10	Validation and Exception Handling
11	UML Diagrams
12	Screenshots and Interface Evidence
13	Testing and Results
14	Limitations and Future Improvements
15	Challenges and Solutions
16	Conclusion
Appendix	Build and Run Instructions

Note: This report describes the implementation present in the submitted project ZIP. Features are not described as implemented unless they are supported by the included source files.

1. Project Overview

The Faculty Leave Management System is a menu-driven C++ application designed to maintain faculty leave-request records. It demonstrates object-oriented programming through separate classes, encapsulated data, inheritance, virtual functions, dynamic memory allocation, file handling, and basic validation. The application provides operations to add, view, search, update, and delete leave requests, process an approval workflow, generate a report, and save or load records.

The source code is organized into header (.h) and implementation (.cpp) files. This modular structure separates responsibilities and makes the program easier to read, test, and maintain.

2. Problem Statement

Managing leave requests manually can make it difficult to find records, correct details, remove outdated entries, and retain information between sessions. A simple computerized system is therefore required to store leave-request details, support basic record operations, demonstrate an approval sequence, and preserve records in a text file.

3. Objectives and Scope

- Develop a menu-driven C++ application for managing leave requests.
- Implement add, display, search, update, and delete operations for leave records.
- Represent faculty, department, leave requests, and approval roles using classes.
- Demonstrate inheritance and runtime polymorphism in the approval process.
- Use dynamic memory allocation and release allocated memory safely.
- Save records to a text file and load them again when the application starts.
- Validate essential fields and handle selected invalid inputs using exceptions.

Scope: the current program manages leave-request records and simulates HOD/Dean approval messages. It is a learning project rather than a full institutional HR platform.

4. Features of the System

Feature	Description
Add record	Accepts a leave request and stores its ID, faculty ID, leave type, dates, and reason.
Display records	Displays the leave requests currently held in memory.
Search record	Looks up a request using its Leave ID.
Update record	Changes details of an existing leave request.
Delete record	Removes a selected leave request from the active list.
Process request	Runs the approval flow using HODApproval and DeanApproval classes.
Generate report	Displays a summary/report of stored leave records.
Save and load	Writes records to and reads records from Data/leave_requests.txt.
Automatic save	Saves current records when the user selects Exit.

5. System Requirements and Tools

- Hardware: a standard laptop or desktop computer; 4 GB RAM or more is a practical recommendation.
- Operating system: Windows.
- Editor: Visual Studio Code.
- Compiler: GNU G++ through the MSYS2 UCRT64 environment.
- Language: C++ with standard library input/output and file-handling facilities.

6. System Design and Program Workflow

The application starts by creating a LeaveSystem object and loading previously saved requests. It then repeatedly displays a menu and performs the selected operation. Selecting Exit saves the current data before the program terminates.

Main workflow

Step	Operation
1	Create the LeaveSystem object. Its constructor allocates an array for leave requests.
2	Load saved records from Data/leave_requests.txt.
3	Display the main menu and read the user's choice.
4	Call the corresponding LeaveSystem operation.
5	For approval processing, use the Approval base class interface with derived approval classes.
6	Allow the user to save or load data from the Save / Load submenu.
7	On Exit, save records and finish the program.
8	The LeaveSystem destructor releases the dynamically allocated array.

Main menu options

Option	Operation
1	Add Record
2	Display Records
3	Search Record
4	Update Record
5	Delete Record
6	Main Transaction / Process
7	Report
8	Save / Load
9	Exit

Data flow

User input is handled by the menu and LeaveSystem methods. LeaveRequest objects hold individual records in memory. FileManager reads and writes the records to the data file. Approval, HODApproval, and DeanApproval represent the approval processing hierarchy. Validation supplies checks for IDs and non-empty text.

7. Classes and Responsibilities

Class	Responsibility
Faculty	Stores faculty ID, name, department, and designation. Provides constructors, accessors/mutators, and display functionality.
Department	Stores department ID and department name, with relevant constructors and member functions.
LeaveRequest	Stores a leave ID, faculty ID, leave type, start date, end date, and reason.
Approval	Base class for approval processing. Stores the request ID/status and declares a virtual processApproval() function.
HODApproval	Derived approval class that overrides processApproval() to show the HOD approval result.
DeanApproval	Derived approval class that overrides processApproval() to show the Dean approval result.
LeaveSystem	Coordinates the menu and record operations. Holds the dynamically allocated LeaveRequest array and its capacity/count.
FileManager	Contains file operations used to save and load leave-request records.
Validation	Provides validation helpers for positive IDs and non-empty text values.
main()	Creates the LeaveSystem object, loads records, dispatches menu choices, and saves data before exit.

The Faculty and Department classes are included as part of the domain model. The current LeaveSystem record operations primarily manage LeaveRequest objects; they do not provide a complete faculty/department database or a user-login subsystem.

8. Object-Oriented Programming Concepts

8.1 Classes and Objects

A class defines data and functions for a type of object. For example, LeaveRequest defines the fields and operations for a single leave request, while the LeaveSystem object manages the collection of requests.

8.2 Encapsulation

The main data fields in classes are private or protected, and public member functions provide controlled access. This helps prevent unrelated code from changing internal values directly.

8.3 Constructors and Destructor

Constructors initialize objects when they are created. The LeaveSystem constructor allocates storage for requests. Its destructor uses delete[] to release that storage when the object is destroyed.

8.4 Inheritance

HODApproval and DeanApproval derive from the Approval base class. The derived classes reuse the base interface and provide their own approval behavior.

8.5 Runtime Polymorphism

The processApproval() member function is virtual. An Approval pointer can refer to a derived approval object, and calling the function invokes the derived override. This demonstrates runtime polymorphism.

8.6 Dynamic Memory Allocation

LeaveSystem stores requests through a dynamically allocated array, conceptually using new LeaveRequest[capacity]. The destructor releases the array using delete[]. The current implementation uses a fixed capacity of 10 requests and does not automatically resize the array.

8.7 Access Specifiers

private limits access to class internals, protected allows access from derived classes, and public exposes operations that other parts of the program may call.

9. File Handling and Data Persistence

The program stores records in Data/leave_requests.txt using pipe-separated fields. The current data file contains records with fields in this order: Leave ID, Faculty ID, leave type, start date, end date, and reason. FileManager is responsible for saving and loading this file.

At startup, main() calls loadData(). The Save / Load menu allows the user to save or reload records manually. When option 9 is selected, the program saves the records before exiting. This enables saved records to be available when the program is launched again, provided the application is run with the project root as its working directory.

Example record format

```
101|1|medical leave|20-10-2026|22-10-2026|medical appointment
102|2|Casual leave|25-10-2026|27-10-2026|Personal work
```

10. Validation and Exception Handling

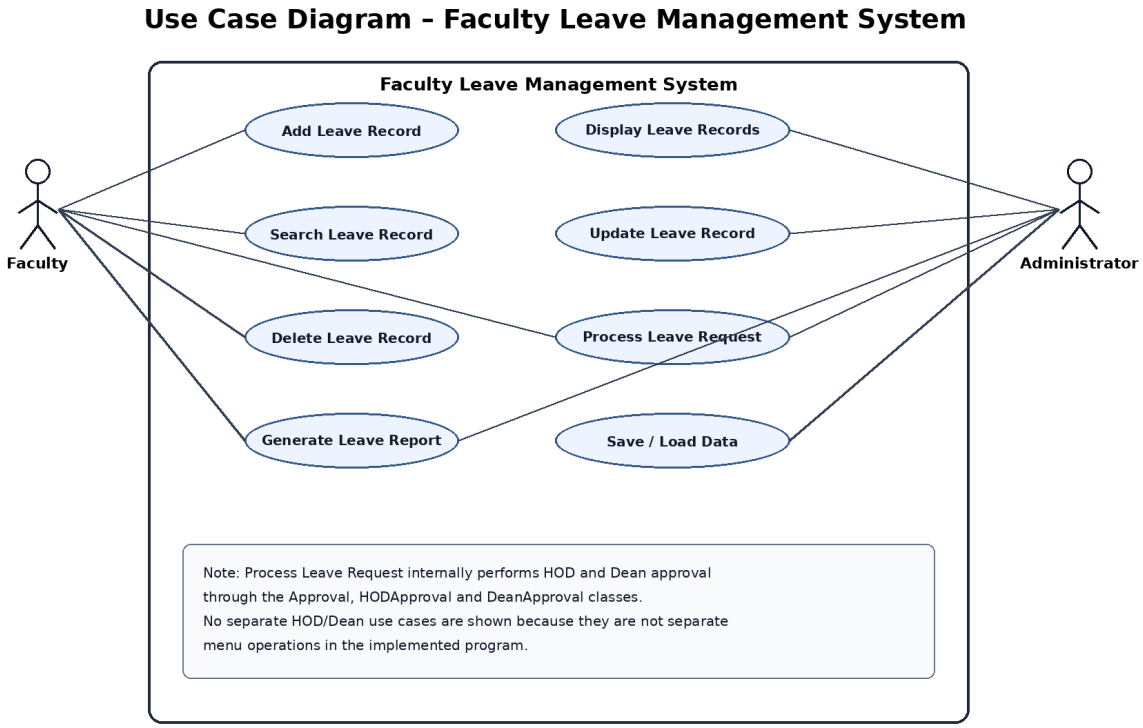
The Validation class checks that IDs are greater than zero and that text values are not empty. Invalid values can raise exceptions, which are handled using try-catch blocks in relevant operations. These checks cover basic data-entry mistakes.

The current validation is intentionally limited. The application does not fully validate date syntax/date ordering, does not prevent duplicate Leave IDs, and does not robustly recover from every non-numeric menu entry. These are useful areas for future improvement.

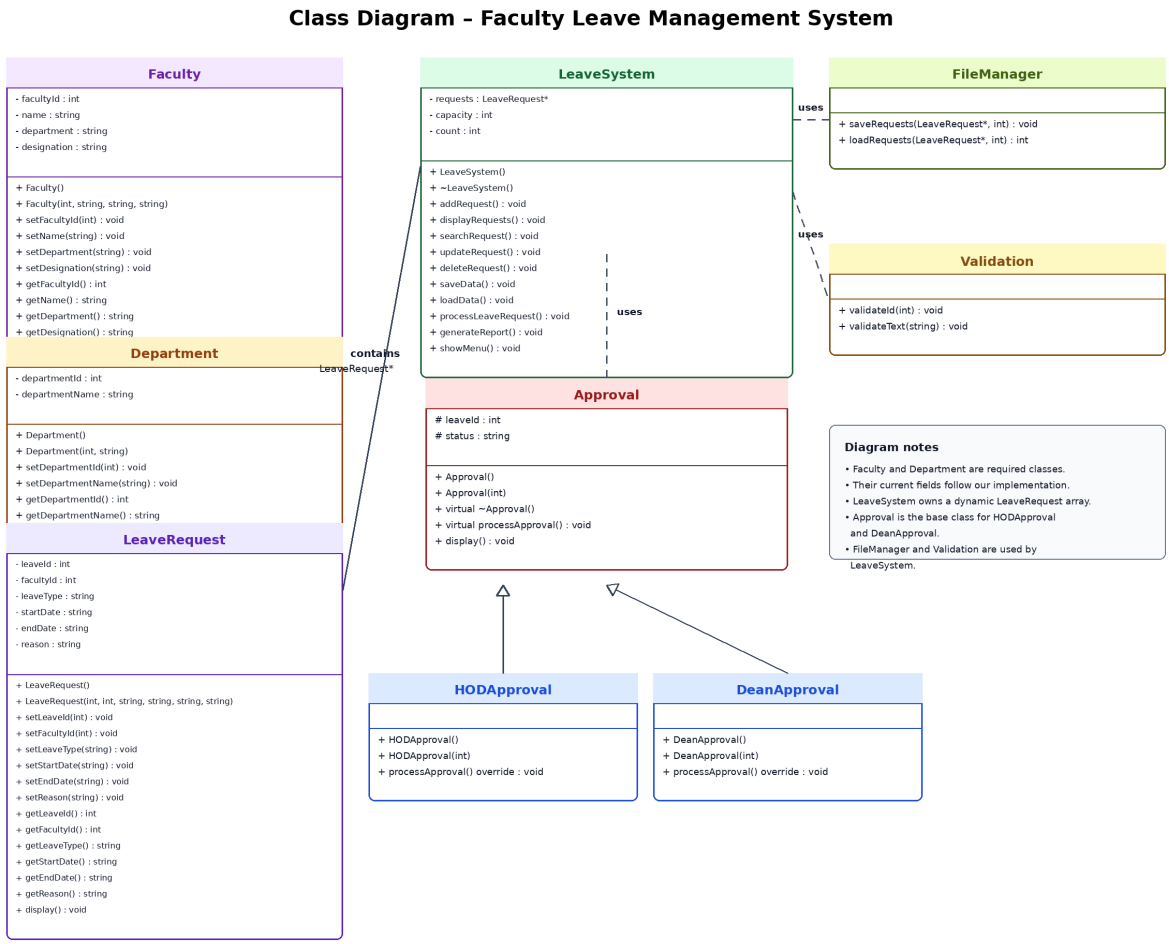
11. UML Diagrams

The following diagrams are included to explain the actors, classes, and message sequence of the application. The UseCaseDiagram(1).png version is used here because it is the more relevant of the two use-case files in the submitted ZIP. The duplicate/outdated diagram should be removed from the final project folder to avoid confusion.

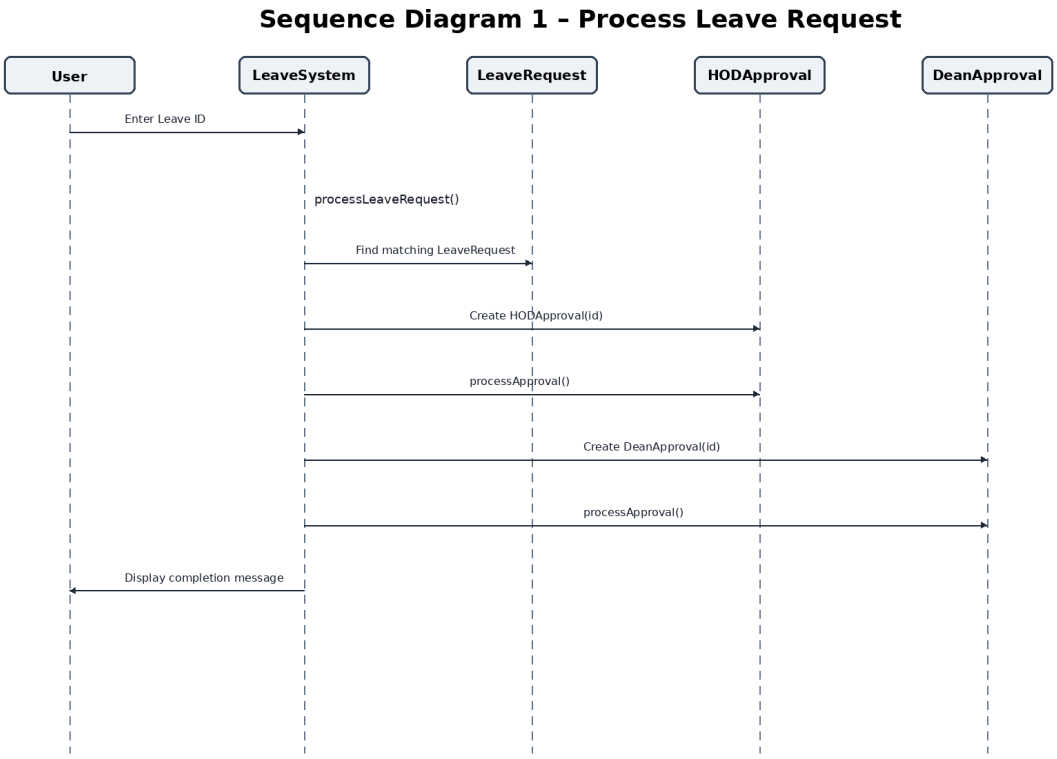
Use-case diagram: illustrates the main user interactions supported by the menu-driven system.



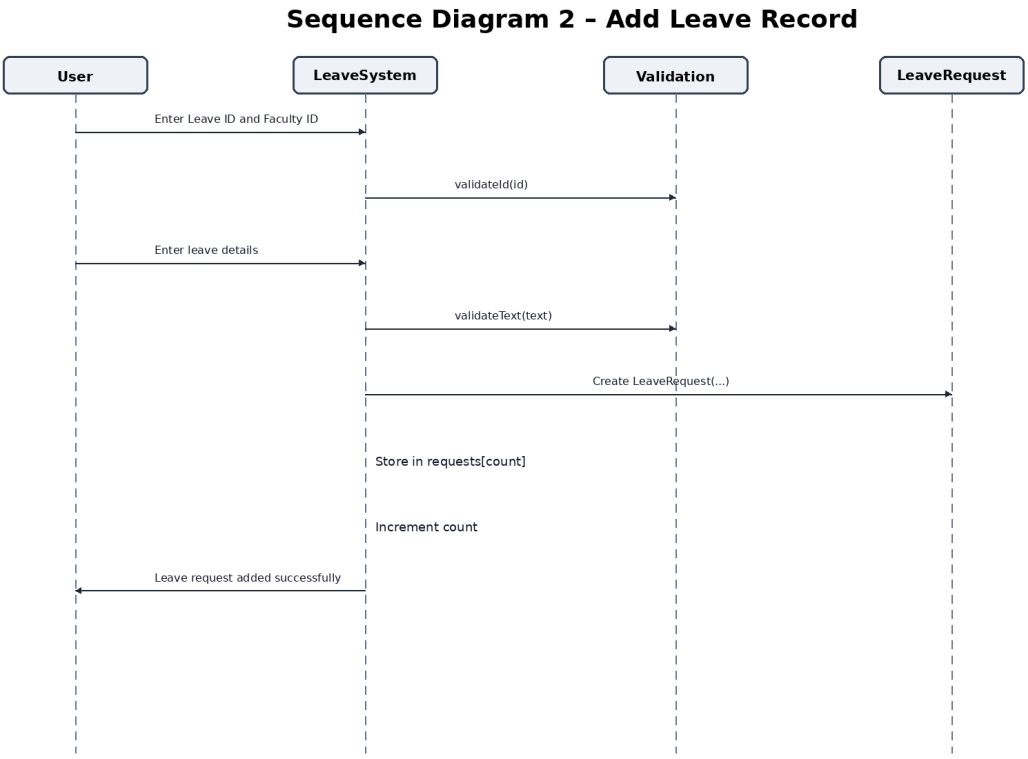
Class diagram: shows the principal classes and their relationships.



Sequence diagram 1: shows a main program interaction sequence.



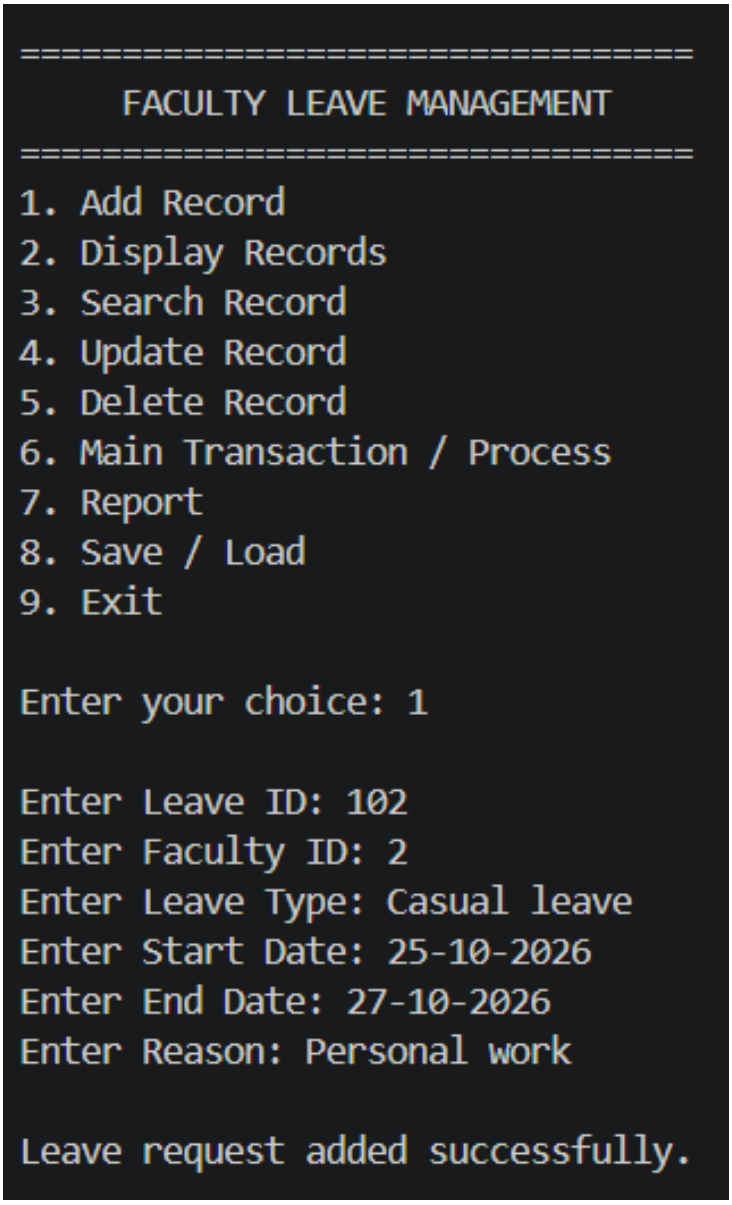
Sequence diagram 2: shows another operation sequence.



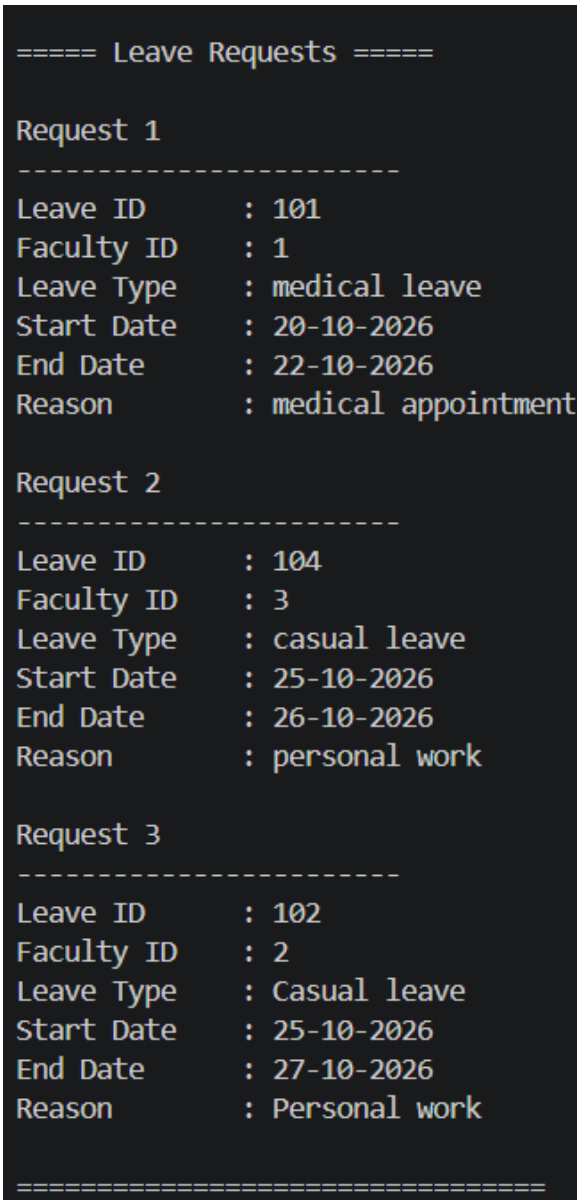
12. Screenshots and Interface Evidence

Screenshots supplied with the project are used as visual evidence for the main operations. The included file named 01_MainMenu.png did not appear to show the actual application main menu during the archive review, so it is not reproduced here as proof of the menu. Replace it with a genuine capture of the program menu before final submission.

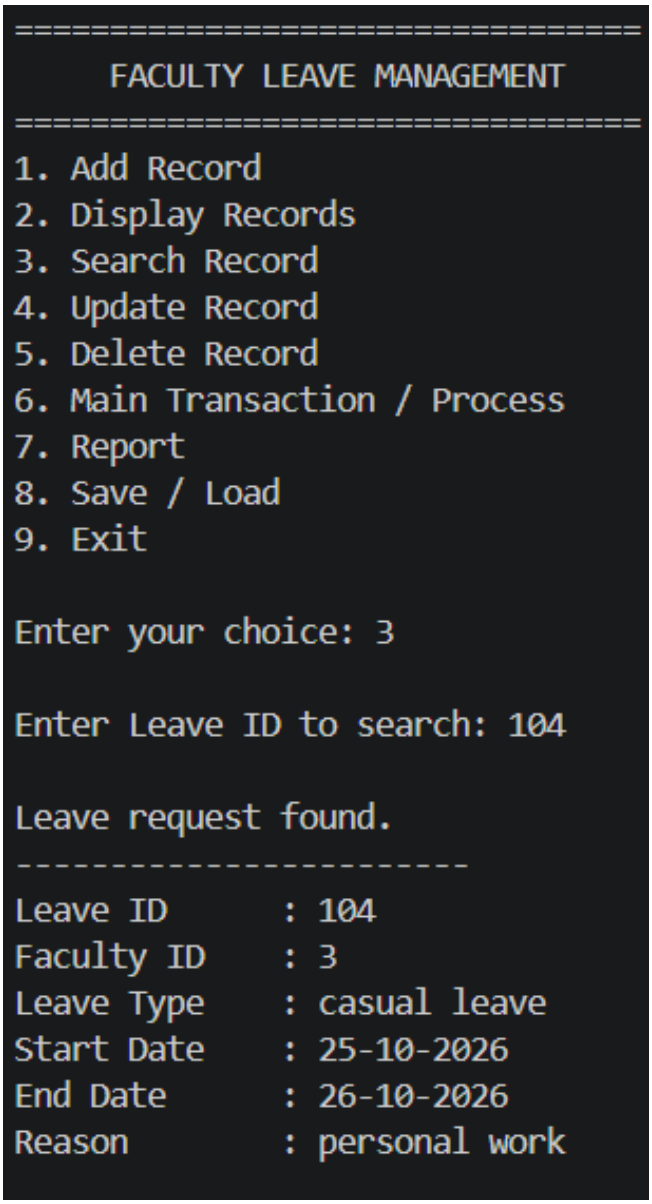
Adding a leave request.



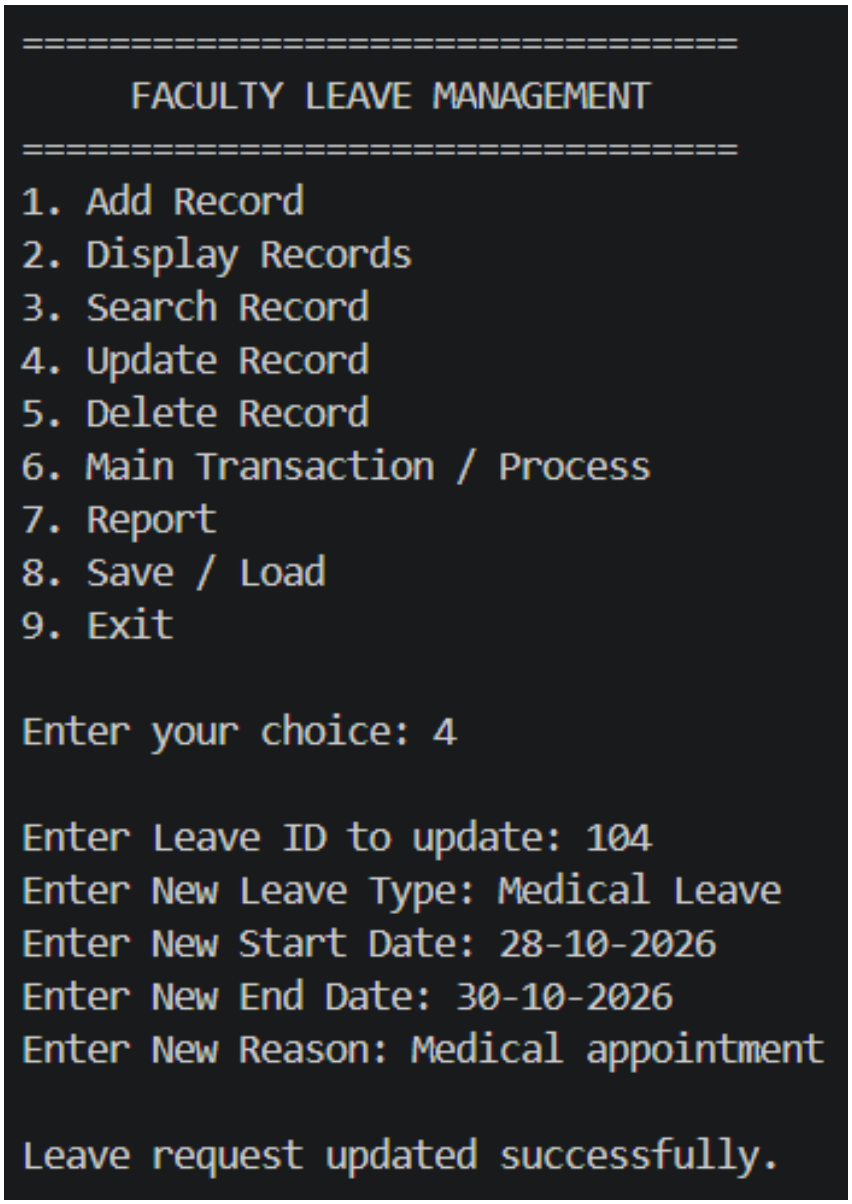
Displaying the saved/in-memory leave requests.



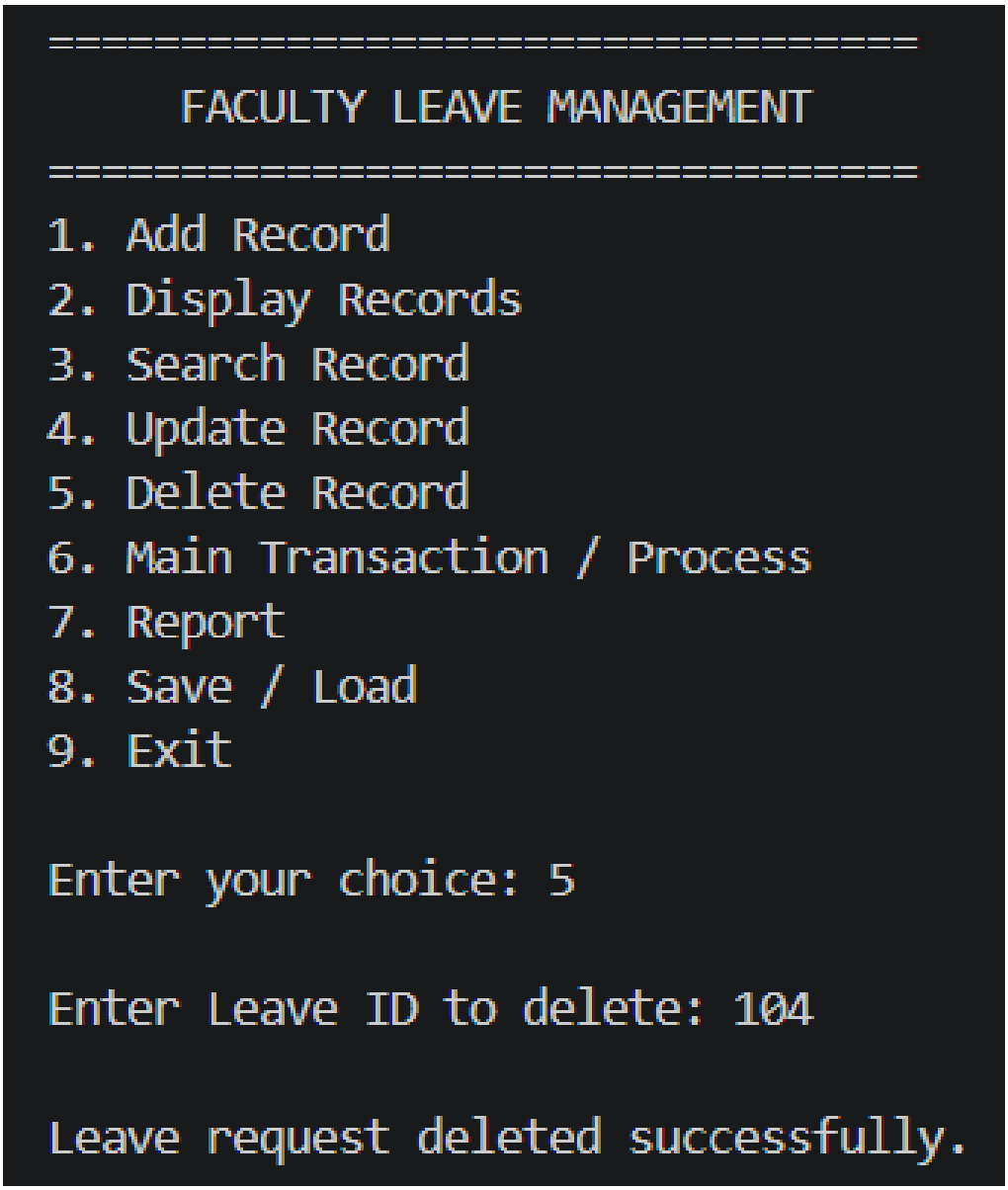
Searching for a leave request.



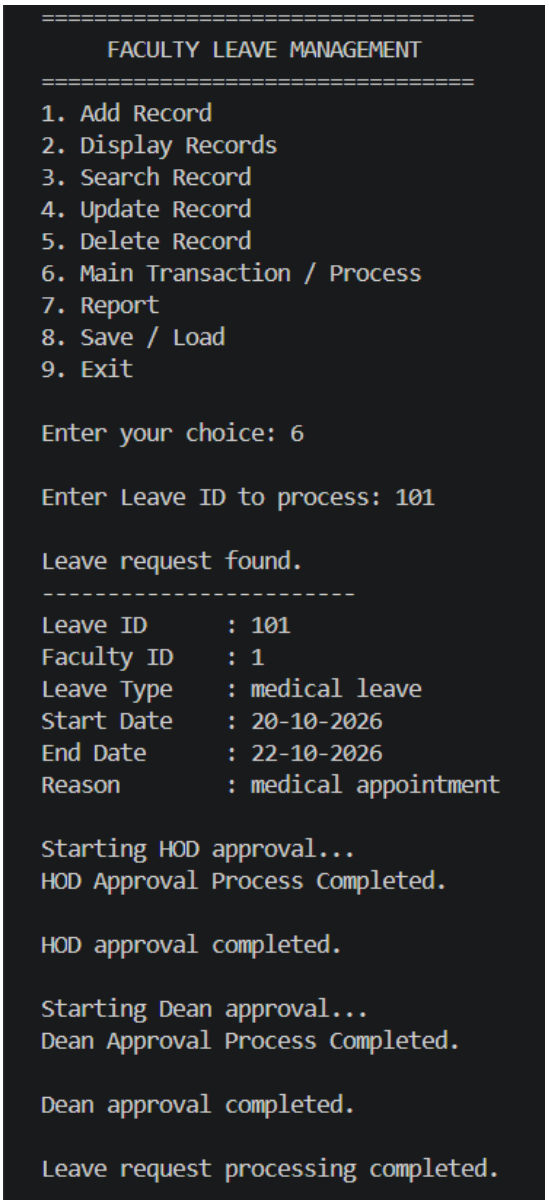
Updating an existing request.



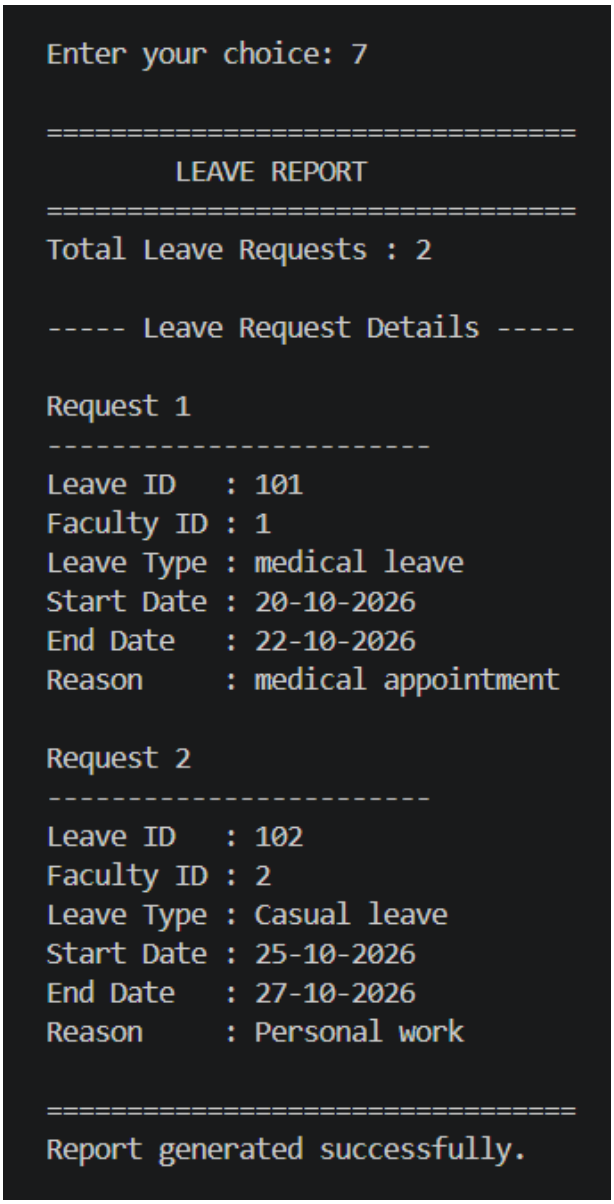
Deleting a selected request.



Processing a request through the approval flow.



Generating the leave report.



```
Enter your choice: 7

================================
        LEAVE REPORT
================================
Total Leave Requests : 2

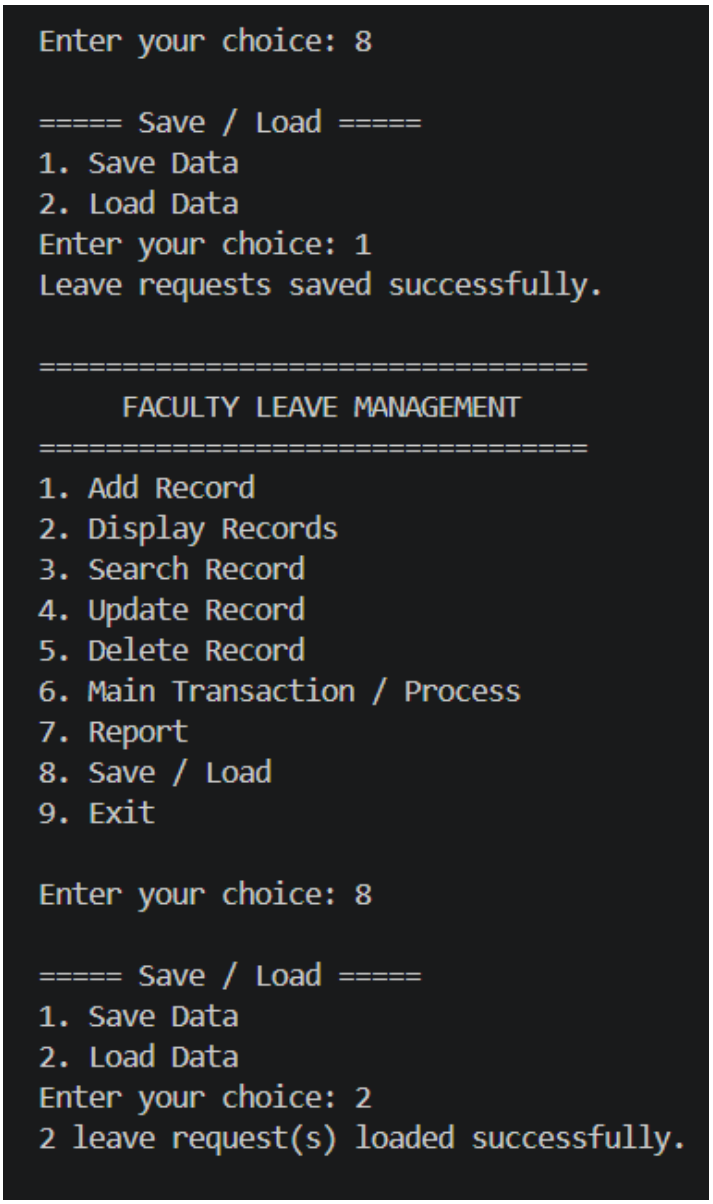
----- Leave Request Details -----

Request 1
------------------------
Leave ID   : 101
Faculty ID : 1
Leave Type : medical leave
Start Date : 20-10-2026
End Date   : 22-10-2026
Reason     : medical appointment

Request 2
------------------------
Leave ID   : 102
Faculty ID : 2
Leave Type : Casual leave
Start Date : 25-10-2026
End Date   : 27-10-2026
Reason     : Personal work

================================
Report generated successfully.
```

Using the Save / Load operation.



Checking that records remain available after restarting.

```
Enter your choice: 9
Leave requests saved successfully.

Thank you for using Faculty Leave Management.
Program exited successfully.
PS C:\Users\HIYA\OneDrive\Desktop\IU2541230522_CPP56_Project> .\FacultyLeaveManagement.exe
2 leave request(s) loaded successfully.

================================
     FACULTY LEAVE MANAGEMENT
================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit

Enter your choice: 2

===== Leave Requests =====

Request 1
------------------------
Leave ID      : 101
Faculty ID    : 1
Leave Type    : medical leave
Start Date    : 20-10-2026
End Date      : 22-10-2026
Reason        : medical appointment

Request 2
------------------------
Leave ID      : 102
Faculty ID    : 2
Leave Type    : Casual leave
```

13. Testing and Results

The following functional checks describe the testing scenarios recorded during project development. These checks focus on the main menu operations and persistence behavior.

Test	Expected behavior	Result/notes
Compile all source files	The source files build into an executable.	Compilation check passed.
Add a valid request	A new request is accepted and can be displayed.	Tested with request records.
Display records	Stored requests are listed.	Tested.
Search existing ID	The matching request is found.	Tested with ID 101 and temporary test records.
Search missing ID	The program reports that the request was not found.	Tested with a missing ID.
Update request	The selected record reflects the changed details.	Tested.
Delete request	The selected record is removed from the active list.	Tested with a temporary record.
Process leave request	HOD and Dean approval messages are produced.	Tested; approval status is not persisted in the leave record.
Generate report	The report option displays a summary of requests.	Tested.
Save and load	Records are written to/read from the text file.	Tested.
Restart persistence	Previously saved requests appear after restart.	Tested.
Empty text validation	An empty text value is rejected.	Tested for an empty leave-type field.
Invalid date / duplicate ID	Invalid dates and duplicate IDs should be rejected.	Not fully implemented; improvement needed.

Testing note: the results above document the checks performed during development and the current limitations identified from source inspection. The project should be re-tested on the submission computer before the final hand-in.

14. Limitations and Future Improvements

- Increase the record capacity dynamically or use a suitable standard container so the system can support more than 10 requests.
- Prevent duplicate Leave IDs and validate all required fields consistently.
- Validate date format and ensure the end date is not earlier than the start date.
- Improve handling of non-numeric menu input so the menu recovers without entering an invalid input state.
- Persist approval status and maintain a clear state for pending, HOD-approved, and Dean-approved requests.
- Connect faculty and department information to leave requests, rather than keeping these classes separate from record operations.
- Add role-based access/login only if required by the project specification.
- Add more detailed error messages for file-open failures and malformed data-file rows.

15. Challenges and Solutions

Challenge	Approach used / possible improvement
Separating the program into files	Classes were separated into .h and .cpp files to make the project modular.
Approval behavior for different roles	A virtual function in Approval is overridden in HODApproval and DeanApproval.
Managing request-array memory	Dynamic allocation is paired with a destructor that releases the array.
Keeping data between runs	FileManager stores records in a text file and loads them on startup.
Invalid input	A Validation class and exception handling cover positive IDs and non-empty text; broader validation remains future work.
Relative data-file path	Run from the project root so Data/leave_requests.txt resolves correctly.

16. Conclusion

The Faculty Leave Management System demonstrates a practical application of C++ object-oriented programming. It supports the main leave-record operations, uses inheritance and runtime polymorphism for an approval workflow, manages dynamically allocated memory, validates selected inputs, and saves records using file handling. The modular design provides a foundation for further improvements, including stronger validation, duplicate-ID prevention, scalable storage, and persistent approval status.

Appendix: Build and Run Instructions

1. Extract the project ZIP and open the IU2541230522_CPP56_Project folder in Visual Studio Code.

2. Open a terminal with the project root as the current working directory.

3. Compile the project using the command below. It includes all implementation files present in Source.

```
g++ Source/main.cpp Source/Faculty.cpp Source/Department.cpp Source/LeaveSystem.cpp
Source/LeaveRequest.cpp Source/FileManager.cpp Source/Validation.cpp Source/Approval.cpp
Source/HODApproval.cpp Source/DeanApproval.cpp -o FacultyLeaveManagement.exe
```

4. Run the executable from the same project-root terminal:

```
.\FacultyLeaveManagement.exe
```

5. Choose option 9 to exit normally; the application saves the current records before closing. Keep the project root as the working directory so the relative Data/ path remains correct.

Submission checklist

- Include all Source .h and .cpp files.
- Include Data/leave_requests.txt.
- Include the four selected UML diagrams.
- Replace Screenshots/01_MainMenu.png with a genuine main-menu screenshot.
- Include screenshots for add, display, search, update, delete, processing, report, save/load, and persistence.
- Include README.txt and this ProjectReport.pdf.
- Keep the executable in Executable/ and ensure it was compiled from the current source.
- Remove the outdated duplicate UseCaseDiagram.png before creating the final ZIP.

End of Report